SQL Views

1. What is a View?

Definition

A **View** is a *virtual table* representing the result of a stored query. It does not store data itself but dynamically shows data from underlying base tables.

Why use Views?

- Simplify complex queries by encapsulating them.
- Provide a security layer by restricting access to specific columns or rows.
- Offer logical data independence by abstracting schema changes.
- Reuse queries easily across multiple applications.

2. Creating Views

Basic syntax to create a view:

Syntax

```
CREATE VIEW view_name AS
SELECT column1, column2, ...
FROM table_name
WHERE condition;
```

Student

ID	name	dept_name
1	Alice	Comp. Sci.
2	Bob	Math
3	Carol	Comp. Sci.
4	Dave	Physics

Example: View of Computer Science Students

```
CREATE VIEW cs_students AS
SELECT ID, name, dept_name
FROM Student
WHERE dept_name = 'Comp. Sci.';
```

You can query this view as:

Usage

```
SELECT * FROM cs_students;
```

3. Views with Joins

Student

ID	name	dept_name
1	Alice	Comp. Sci.
2	Bob	Math
3	Carol	Comp. Sci.
4	Dave	Physics

Takes

ID	course_id	grade
1	CS101	A
1	MA101	B
2	MA101	A
3	CS101	B
4	PH101	A

Concept

Views can be created using joins to combine data from multiple tables into one virtual table.

Example: View for Students and Their Courses

```
CREATE VIEW student_courses AS
SELECT s.ID, s.name, t.course_id, t.grade
FROM Student s
JOIN Takes t ON s.ID = t.ID;
```

Query example:

Usage

```
SELECT name, course_id, grade
FROM student_courses
WHERE grade = 'A';
```

4. Example Views

View of instructors without their salary

```
CREATE VIEW faculty AS
SELECT ID, name, dept_name
FROM instructor;
```

Find all instructors in the Biology department using the view:

Query using view

```
SELECT name
FROM faculty
WHERE dept_name = 'Biology';
```

Create a view of department salary totals:

Department salary totals

```
CREATE VIEW departments_total_salary(dept_name, total_salary) AS
SELECT dept_name, SUM(salary)
FROM instructor
GROUP BY dept_name;
```

5. Views Defined Using Other Views

Physics Fall 2009 courses

```
CREATE VIEW physics_fall_2009 AS
SELECT course.course_id, sec_id, building, room_number
FROM course, section
WHERE course.course_id = section.course_id
  AND course.dept_name = 'Physics'
  AND section.semester = 'Fall'
  AND section.year = '2009';
```

Physics Fall 2009 courses in Watson building

```
CREATE VIEW physics_fall_2009_watson AS
SELECT course_id, room_number
FROM physics_fall_2009
WHERE building = 'Watson';
```

View Expansion

View expansion is the process of replacing views used in a view definition by their underlying query expressions recursively until only base tables remain.
Note: This process terminates if views are not recursive.

6. Materialized Views

Definition

Materialized views store the actual result of a query physically. They improve performance but require maintenance to stay consistent with underlying data.

View Maintenance

Materialized views need to be updated whenever base tables change. This update (view maintenance) can be:

- Immediate (eager) — updated on every change.
- Lazy — updated when accessed.
- Periodic — updated at set intervals, potentially causing stale data.

DBAs may control maintenance strategy.

7. Updating Views

Challenges with Updating Views

Updating, inserting, or deleting data via views is complex because changes must translate back to base tables. This is not always possible or unambiguous.

Example: Inserting a tuple into the `faculty` view

Insert into view

```
INSERT INTO faculty VALUES ('30765', 'Green', 'Music');
```

Possible outcomes:

- Reject the insertion with an error.
- Insert the tuple into the `instructor` base table, possibly with NULLs for omitted columns.

Some updates cannot be translated uniquely, e.g.:

Ambiguous update

```
CREATE VIEW instructor_info AS
SELECT ID, name, building
FROM instructor, department
WHERE instructor.dept_name = department.dept_name;

INSERT INTO instructor_info VALUES ('69987', 'White', 'Taylor');
```

Question: Which department should the new instructor belong to if multiple departments are in 'Taylor'? What if none?

8. Restrictions on Updatable Views

Most SQL implementations only allow updates on **simple views** that:

- Are based on a single table.
- Use only attribute names (no expressions, aggregates, or DISTINCT).

- Contain no GROUP BY or HAVING clauses.
- Do not involve joins or UNION.
- Allow omitted attributes to be NULL or non-primary key.

Example of an updatable view:

Updatable view

```
CREATE VIEW history_instructors AS
SELECT *
FROM instructor
WHERE dept_name = 'History';
```

Inserting a tuple violating the view's condition may be allowed unless:

WITH CHECK OPTION

Views can be created with `WITH CHECK OPTION` to ensure that inserted or updated rows satisfy the view's WHERE condition.
If not satisfied, the insertion/update is rejected.

9. Summary

Summary

- Views simplify complex queries and provide abstraction and security.
- Views can be defined on base tables or other views.
- Materialized views store data physically and require maintenance.
- Updating views is limited to simple cases and can be ambiguous.
- Use `WITH CHECK OPTION` to enforce integrity on views.